

Monisha Barns

Sales retail management system

Feb 14, 2026



Overview

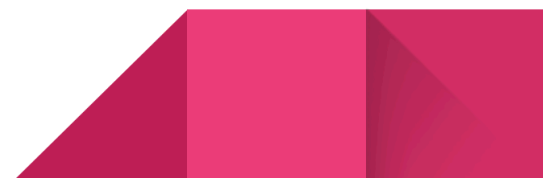
Design and implement a Smart Retail Management System (SRMS) using SQL Server that manages products, categories, suppliers, customers, orders, employees, warehouses, and payments.

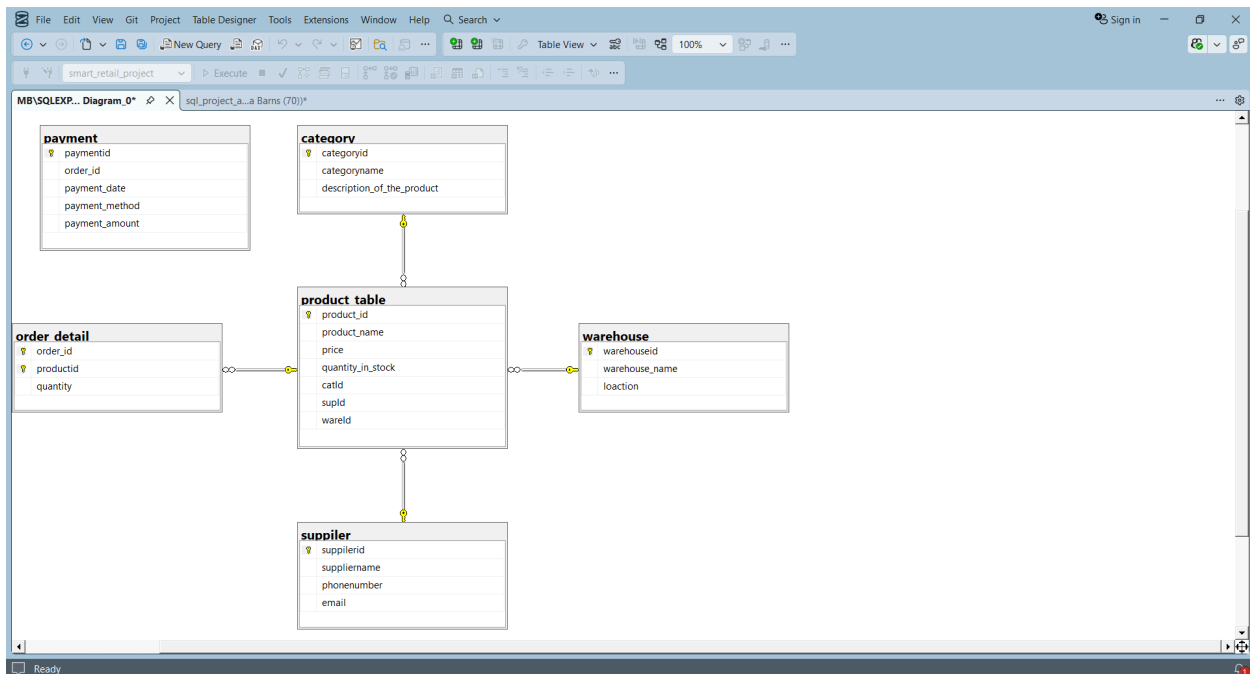
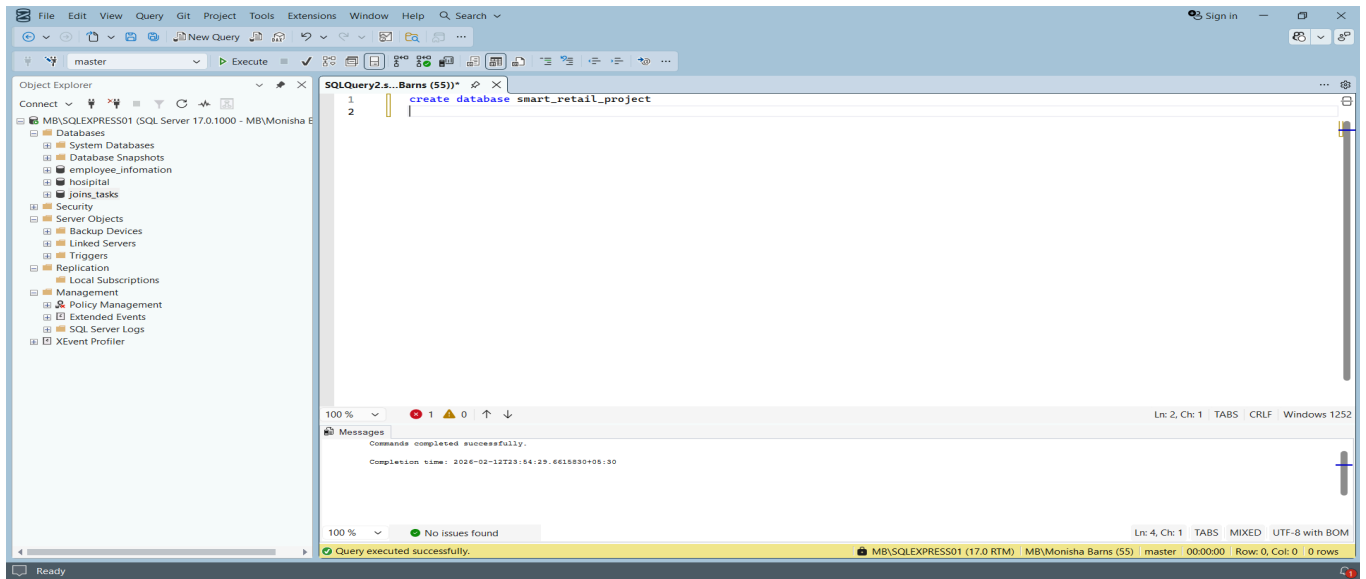
Goals

A retail company wants to build a centralized database system to manage: Product and category information, Supplier and warehouse stock details, Customer orders and payments, Employees who process orders, Sales and inventory reports

Milestones

SECTION A: Database Design



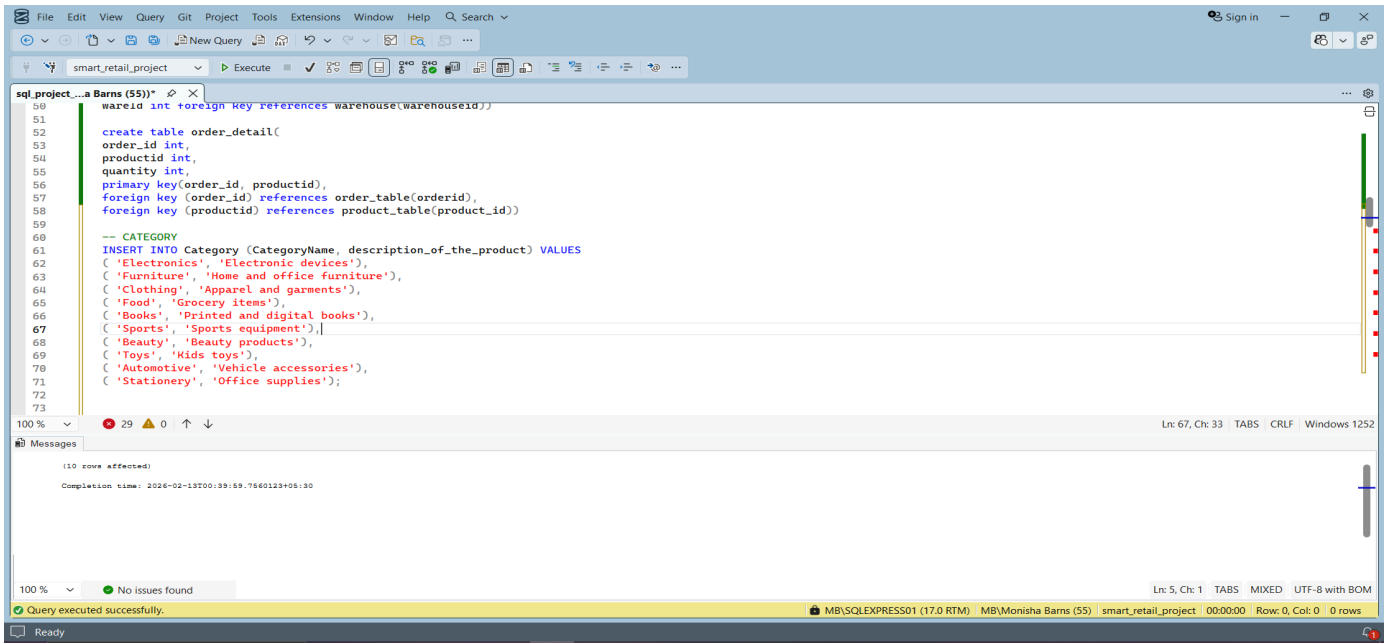


Section B – Table Creation & Constraints:

```
File Edit View Query Git Project Tools Extensions Window Help Search
smart_retail_project Execute
SQLQuery2.s...Barns (55)*
1 create database smart_retail_project
2
3 use smart_retail_project
4
5 create table category(
6 categoryid int primary key identity(1,1),
7 categoryname varchar(20),
8 description_of_the_product varchar(50))
9
10 create table supplier(
11 supplierid int primary key identity(201,1),
12 suppliername varchar(20),
13 phonenumber bigint,
14 email varchar(50))
15
16 create table customer(
17 customerid int primary key identity(101,1),
18 customername varchar(20),
19 phoneno bigint,
20 email_id varchar(50),
21 adress varchar(60))
22
23 create table order_table(
24 orderid int primary key identity(10,1),
25 orderdate date,
26 total_amount bigint,
27 cid int foreign key references customer(customerid))
28
29 create table warehouse(
30 warehouseid int primary key identity(501,1),
31 warehouse_name varchar(30),
32 loaction varchar(30))
33
100 % 1 0
Ln: 54, Ch: 15 TABS CRLF Windows 1252
Messages
100 % No issues found
Ln: 4, Ch: 1 TABS MIXED UTF-8 with BOM
Query executed successfully. MB\SQLEXPRESS01 (17.0 RTM) MB\Monisha Barns (55) smart_retail_project 00:00:00 Row: 0, Col: 0 0 rows
Ready
```

```
File Edit View Query Git Project Tools Extensions Window Help Search
smart_retail_project Execute
SQLQuery2.s...Barns (55)*
33
34
35 create table payment(
36 paymentid int primary key identity(401,1),
37 order_id int foreign key references order_table(orderid),
38 payment_date date,
39 payment_method varchar(20),
40 payment_amount money check(payment_amount>0))
41
42
43 create table product_table(
44 product_id int primary key identity(301,1),
45 product_name varchar(20),
46 price money check(price>0),
47 quantity_in_stock int,
48 catId int foreign key references customer(customerid),
49 supId int foreign key references supplier(supplierid),
50 wareId int foreign key references warehouse(warehouseid))
51
52 create table order_detail(
53 order_id int,
54 productid int,
55 primary key(order_id, productid),
56 foreign key (order_id) references order_table(orderid),
57 foreign key (productid) references product_table(product_id))
58
100 % 1 0
Ln: 54, Ch: 15 TABS CRLF Windows 1252
Messages
100 % No issues found
Ln: 4, Ch: 1 TABS MIXED UTF-8 with BOM
Query executed successfully. MB\SQLEXPRESS01 (17.0 RTM) MB\Monisha Barns (55) smart_retail_project 00:00:00 Row: 0, Col: 0 0 rows
Ready
```

Section C – Data Insertion



```
sql_project...a Barns (55)*
50 warehouse int foreign key references warehouse(warehouseid))
51
52 create table order_detail(
53     order_id int,
54     productid int,
55     quantity int,
56     primary key(order_id, productid),
57     foreign key (order_id) references order_table(orderid),
58     foreign key (productid) references product_table(product_id))
59
60 -- CATEGORY
61 INSERT INTO Category (CategoryName, description_of_the_product) VALUES
62 ( 'Electronics', 'Electronic devices'),
63 ( 'Furniture', 'Home and office furniture'),
64 ( 'Clothing', 'Apparel and garments'),
65 ( 'Food', 'Grocery items'),
66 ( 'Books', 'Printed and digital books'),
67 ( 'Sports', 'Sports equipment'),
68 ( 'Beauty', 'Beauty products'),
69 ( 'Toys', 'Kids toys'),
70 ( 'Automotive', 'Vehicle accessories'),
71 ( 'Stationery', 'Office supplies');
72
73
```

100 % 29 0 100 %

Messages

(10 rows affected)

Completion time: 2024-02-13T00:39:59.7560123+05:30

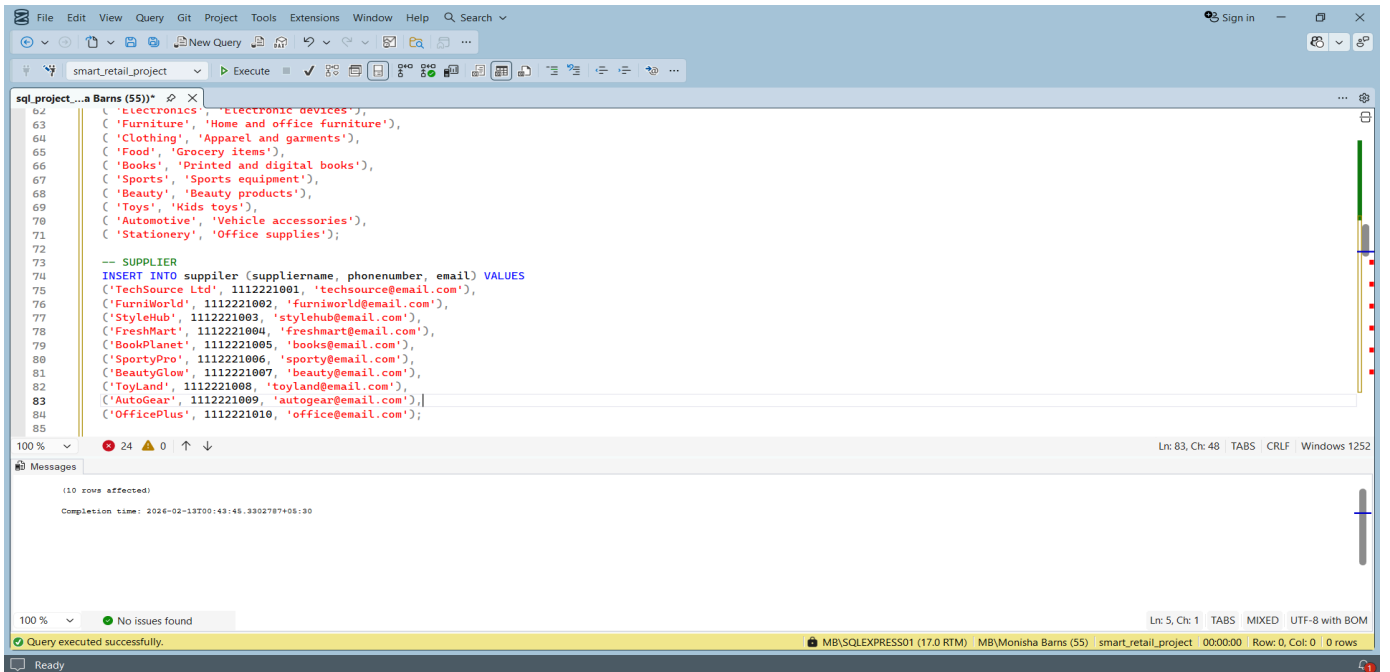
Ln: 67, Ch: 33 TABS CRLF Windows 1252

100 % No issues found

Ln: 5, Ch: 1 TABS MIXED UTF-8 with BOM

Query executed successfully. MB\SQLEXPRESS01 (17.0 RTM) MB\Monisha Barns (55) smart_retail_project 00:00:00 Row: 0, Col: 0 0 rows

Ready



```
sql_project...a Barns (55)*
62 ( 'Electronics', 'Electronic devices'),
63 ( 'Furniture', 'Home and office furniture'),
64 ( 'Clothing', 'Apparel and garments'),
65 ( 'Food', 'Grocery items'),
66 ( 'Books', 'Printed and digital books'),
67 ( 'Sports', 'Sports equipment'),
68 ( 'Beauty', 'Beauty products'),
69 ( 'Toys', 'Kids toys'),
70 ( 'Automotive', 'Vehicle accessories'),
71 ( 'Stationery', 'Office supplies');
72
73 -- SUPPLIER
74 INSERT INTO supplier (suppliername, phonenumber, email) VALUES
75 ('TechSource Ltd', 1112221001, 'techsource@email.com'),
76 ('FurniWorld', 1112221002, 'furniworld@email.com'),
77 ('StyleHub', 1112221003, 'stylehub@email.com'),
78 ('FreshMart', 1112221004, 'freshmart@email.com'),
79 ('BookPlanet', 1112221005, 'books@email.com'),
80 ('SportyPro', 1112221006, 'sporty@email.com'),
81 ('BeautyGlow', 1112221007, 'beauty@email.com'),
82 ('ToyLand', 1112221008, 'toyland@email.com'),
83 ('AutoGear', 1112221009, 'autogear@email.com'),
84 ('OfficePlus', 1112221010, 'office@email.com');
85
```

100 % 24 0 100 %

Messages

(10 rows affected)

Completion time: 2024-02-13T00:43:45.3302787+05:30

Ln: 83, Ch: 48 TABS CRLF Windows 1252

100 % No issues found

Ln: 5, Ch: 1 TABS MIXED UTF-8 with BOM

Query executed successfully. MB\SQLEXPRESS01 (17.0 RTM) MB\Monisha Barns (55) smart_retail_project 00:00:00 Row: 0, Col: 0 0 rows

Ready

```
File Edit View Query Git Project Tools Extensions Window Help Search
smart_retail_project Execute
sql_project...a Barns (55)*
76 ('Turn1morto', 1112221002, 'turn1morto@email.com'),
77 ('StyleHub', 1112221003, 'stylehub@email.com'),
78 ('FreshMart', 1112221004, 'freshmart@email.com'),
79 ('BookPlanet', 1112221005, 'books@email.com'),
80 ('SportyPro', 1112221006, 'sporty@email.com'),
81 ('BeautyGlow', 1112221007, 'beauty@email.com'),
82 ('ToyLand', 1112221008, 'toyland@email.com'),
83 ('AutoGear', 1112221009, 'autogear@email.com'),
84 ('OfficePlus', 1112221010, 'office@email.com');
85
86 -- WAREHOUSE
87 INSERT INTO warehouse (warehouse_name, loaction) VALUES
88 ('Central Warehouse', 'New York'),
89 ('West Storage', 'Los Angeles'),
90 ('East Depot', 'Boston'),
91 ('South Hub', 'Texas'),
92 ('North Storage', 'Chicago'),
93 ('Metro Warehouse', 'Miami'),
94 ('Prime Depot', 'Seattle'),
95 ('Quick Storage', 'Denver'),
96 ('City Warehouse', 'Atlanta'),
97 ('Global Depot', 'Phoenix');
98
99
100 % 22 0
Ln: 99, Ch: 1 TABS CRLF Windows 1252
Messages
(10 rows affected)
Completion time: 2026-02-13T00:48:03.9626771+05:30
100 % No issues found
Ln: 5, Ch: 1 TABS MIXED UTF-8 with BOM
Query executed successfully. MB\SQL EXPRESS01 (17.0 RTM) MB\Monisha Barns (55) smart_retail_project 00:00:00 Row: 0, Col: 0 0 rows
Ready
```

```
File Edit View Query Git Project Tools Extensions Window Help Search
smart_retail_project Execute
sql_project...a Barns (55)*
89 ('West Storage', 'Los Angeles'),
90 ('East Depot', 'Boston'),
91 ('South Hub', 'Texas'),
92 ('North Storage', 'Chicago'),
93 ('Metro Warehouse', 'Miami'),
94 ('Prime Depot', 'Seattle'),
95 ('Quick Storage', 'Denver'),
96 ('City Warehouse', 'Atlanta'),
97 ('Global Depot', 'Phoenix');
98
99 -- PRODUCT
100 INSERT INTO product_table (product_name, price, quantity_in_stock) VALUES
101 ('Laptop', 800, 50),
102 ('Office Chair', 120, 100),
103 ('T-Shirt', 25, 200),
104 ('Rice Bag', 40, 150),
105 ('Novel Book', 15, 120),
106 ('Football', 30, 80),
107 ('Face Cream', 18, 90),
108 ('Toy Car', 22, 110),
109 ('Car Battery', 150, 60),
110 ('Notebook Pack', 10, 300);
111
112
100 % 14 0
Ln: 108, Ch: 22 TABS CRLF Windows 1252
Messages
(10 rows affected)
Completion time: 2026-02-13T00:47:27.0200669+05:30
100 % No issues found
Ln: 5, Ch: 1 TABS MIXED UTF-8 with BOM
Query executed successfully. MB\SQL EXPRESS01 (17.0 RTM) MB\Monisha Barns (55) smart_retail_project 00:00:00 Row: 0, Col: 0 0 rows
Ready
```

```
File Edit View Query Git Project Tools Extensions Window Help Search
smart_retail_project Execute
sql_project...a Barns (55)*
102 ('Office Chair', 120, 100),
103 ('T-Shirt', 25, 200),
104 ('Rice Bag', 40, 150),
105 ('Novel Book', 15, 120),
106 ('Football', 30, 80),
107 ('Face Cream', 18, 90),
108 ('Toy Car', 22, 110),
109 ('Car Battery', 150, 60),
110 ('Notebook Pack', 10, 300);
111
112 -- CUSTOMER
113 INSERT INTO customer (customername, phoneno, email_id, address) VALUES
114 ('John Smith', 5550001, 'john@email.com', 'NV'),
115 ('Sarah Lee', 5550002, 'sarah@email.com', 'LA'),
116 ('David Kim', 5550003, 'david@email.com', 'TX'),
117 ('Emma Brown', 5550004, 'emma@email.com', 'FL'),
118 ('Michael Chen', 5550005, 'michael@email.com', 'WA'),
119 ('Olivia Davis', 5550006, 'olivia@email.com', 'NV'),
120 ('Daniel Wilson', 5550007, 'daniel@email.com', 'IL'),
121 ('Sophia Moore', 5550008, 'sophia@email.com', 'GA'),
122 ('James Taylor', 5550009, 'james@email.com', 'AZ'),
123 ('Ava Martinez', 5550010, 'ava@email.com', 'CO');
124
125
100% 11 0 11
Messages
(10 rows affected)
Completion time: 2026-02-13T00:50:08.853712+05:30
100% No issues found
Ln: 123, Ch: 50 TABS CRLF Windows 1252
Query executed successfully. MB/SQLEXPRESS01 (17.0 RTM) MB/Monisha Barns (55) smart_retail_project 00:00:00 Row: 0, Col: 0 0 rows
Ready
```

```
File Edit View Query Git Project Tools Extensions Window Help Search
smart_retail_project Execute
sql_project...a Barns (55)*
114 ('John Smith', 5550001, 'john@email.com', 'NV'),
115 ('Sarah Lee', 5550002, 'sarah@email.com', 'LA'),
116 ('David Kim', 5550003, 'david@email.com', 'TX'),
117 ('Emma Brown', 5550004, 'emma@email.com', 'FL'),
118 ('Michael Chen', 5550005, 'michael@email.com', 'WA'),
119 ('Olivia Davis', 5550006, 'olivia@email.com', 'NV'),
120 ('Daniel Wilson', 5550007, 'daniel@email.com', 'IL'),
121 ('Sophia Moore', 5550008, 'sophia@email.com', 'GA'),
122 ('James Taylor', 5550009, 'james@email.com', 'AZ'),
123 ('Ava Martinez', 5550010, 'ava@email.com', 'CO');
124
125 -- ORDERS
126 INSERT INTO order_table (OrderDate, total_amount) VALUES
127 ('2026-01-01', 800),
128 ('2026-01-02', 120),
129 ('2026-01-03', 50),
130 ('2026-01-04', 40),
131 ('2026-01-05', 30),
132 ('2026-01-06', 18),
133 ('2026-01-07', 22),
134 ('2026-01-08', 150),
135 ('2026-01-09', 10),
136 ('2026-01-10', 25);
137
100% 5 0 5
Messages
(10 rows affected)
Completion time: 2026-02-13T00:52:35.1397708+05:30
100% No issues found
Ln: 135, Ch: 20 TABS CRLF Windows 1252
Query executed successfully. MB/SQLEXPRESS01 (17.0 RTM) MB/Monisha Barns (55) smart_retail_project 00:00:00 Row: 0, Col: 0 0 rows
Ready
```

```
File Edit View Query Git Project Tools Extensions Window Help Search
smart_retail_project Execute
sql_project...a Barns (55)*
131
132 ('2026-01-06', 18),
133 ('2026-01-07', 22),
134 ('2026-01-08', 150),
135 ('2026-01-09', 10),
136 ('2026-01-10', 25);
137
138
139 select * from product_table
140 -- ORDER_DETAIL
141 INSERT INTO order_detail(order_id, productid, quantity) VALUES
142 (10, 301, 1),
143 (11, 302, 1),
144 (12, 303, 2),
145 (13, 304, 1),
146 (14, 305, 1),
147 (15, 306, 1),
148 (16, 307, 1),
149 (17, 308, 1),
150 (18, 309, 1),
151 (19, 310, 1);
152
100% 4 0 1 2 Messages
(10 rows affected)
Completion time: 2026-02-13T01:03:16.1761067+05:30
100% No issues found Ln: 145, Ch: 12 TABS CRLF Windows 1252
Query executed successfully. MB/SQLEXPRESS01 (17.0 RTM) MB/Monisha Barns (55) smart_retail_project 00:00:00 Row: 1, Col: 1 0 rows
Ready
```

```
File Edit View Query Git Project Tools Extensions Window Help Search
smart_retail_project Execute
sql_project...a Barns (55)*
146 (14, 305, 1),
147 (15, 306, 1),
148 (16, 307, 1),
149 (17, 308, 1),
150 (18, 309, 1),
151 (19, 310, 1);
152
153 -- PAYMENT
154 INSERT INTO payment(payment_date, payment_method, payment_amount) VALUES
155 ('2026-01-01', 'Credit Card', 800),
156 ('2026-01-02', 'Cash', 120),
157 ('2026-01-03', 'Debit Card', 50),
158 ('2026-01-04', 'Cash', 40),
159 ('2026-01-05', 'Credit Card', 30),
160 ('2026-01-06', 'Online', 18),
161 ('2026-01-07', 'Debit Card', 22),
162 ('2026-01-08', 'Credit Card', 150),
163 ('2026-01-09', 'Cash', 10),
164 ('2026-01-10', 'Online', 25);
165
166
167
100% No issues found Ln: 165, Ch: 1 TABS CRLF Windows 1252
Query executed successfully. MB/SQLEXPRESS01 (17.0 RTM) MB/Monisha Barns (55) smart_retail_project 00:00:00 Row: 1, Col: 1 0 rows
Ready
```

Section D – Basic SQL Queries:

The screenshot displays the Microsoft SQL Server Enterprise Manager interface. The left pane shows the 'Object Explorer' with the 'smart_retail_project' database selected. The main query window shows a SQL script with the following content:

```
177      ('2026-01-06', 'Online', 18),
178      ('2026-01-07', 'Debit Card', 22),
179      ('2026-01-08', 'Credit Card', 150),
180      ('2026-01-09', 'Cash', 10),
181      ('2026-01-10', 'Online', 25);
182
183
184
185      --section D
186      --Display all products
187      select product_id ,product_name from product_table
188
189      --Display products whose price is greater than 1000.
190      select product_id, product_name, price from product_table where price>1000
191
192      --Find customers from a particular city(eg.LA)
193      select customerid, customername ,address from customer where address='LA'
194
195      --Update the price of a product.id=309
196      select * from customer
197      update product_table set price=1010 where product_id=309
198
199      --Delete customers who have not placed any orders
```

The 'Results' pane shows the output of the query executed on line 187, displaying a table with two columns: 'product_id' and 'product_name'. The results are as follows:

product_id	product_name
301	Laptop
302	Office Chair
303	T-Shirt
304	Rice Bag
305	Novel Book
306	Football
307	Face Cream
308	Toy Car
309	Car Battery
310	Notebook Pa...

The status bar at the bottom indicates 'Query executed successfully.' and 'MB\SQLEXPRESS01 (17.0 RTM) MB\Monisha Barns (70) smart_retail_project 00:00:00 Row: 1, Col: 1 10 rows'.

SQL Server Enterprise Edition (64-bit) - smart_retail_project

Object Explorer: Connect to MB\SQLEXPRESS01 (SQL Serv... Databases Security Server Objects Replication Management XEvent Profiler

Query Editor (SQL): sql_project...a Barns (70)

```
179 ('2026-01-06', 'Online', 18),
180 ('2026-01-07', 'Debit Card', 22),
181 ('2026-01-08', 'Credit Card', 150),
182 ('2026-01-09', 'Cash', 10),
183 ('2026-01-10', 'Online', 25);
184
185 --section D
186 --Display all products
187 select product_id ,product_name from product_table
188
189 --Display products whose price is greater than 1000.
190 select product_id, product_name, price from product_table where price>1000
191
192 --Find customers from a particular city(eg.LA)
193 select customerid, customername ,adress from customer where adress='LA'
194
195 --Update the price of a product.id=309
196 select * from customer
197 update product_table set price=1010 where product_id=309
198
199 --Delete customers who have not placed any orders
```

Results: product_id product_name price
1 309 Car Battery 1010.00

Query executed successfully. MB\SQLEXPRESS01 (17.0 RTM) MB\Monisha Barns (70) smart_retail_project 00:00:00 Row: 1, Col: 1 1 rows

SQL Server Enterprise Edition (64-bit) - smart_retail_project

Object Explorer: Connect to MB\SQLEXPRESS01 (SQL Serv... Databases Security Server Objects Replication Management XEvent Profiler

Query Editor (SQL): sql_project...a Barns (70)

```
179 ('2026-01-06', 'Online', 18),
180 ('2026-01-07', 'Debit Card', 22),
181 ('2026-01-08', 'Credit Card', 150),
182 ('2026-01-09', 'Cash', 10),
183 ('2026-01-10', 'Online', 25);
184
185 --section D
186 --Display all products
187 select product_id ,product_name from product_table
188
189 --Display products whose price is greater than 1000.
190 select product_id, product_name, price from product_table where price>1000
191
192 --Find customers from a particular city(eg.LA)
193 select customerid, customername ,adress from customer where adress='LA'
194
195 --Update the price of a product.id=309
196 select * from customer
197 update product_table set price=1010 where product_id=309
198
199 --Delete customers who have not placed any orders
```

Results: customerid customername adress
1 102 Sarah Lee LA

Query executed successfully. MB\SQLEXPRESS01 (17.0 RTM) MB\Monisha Barns (70) smart_retail_project 00:00:00 Row: 1, Col: 1 1 rows

Object Explorer

Connect ▼ MB\SQLEXPRESS01 (SQL Serv

Databases

Security

Server Objects

Replication

Management

XEvent Profiler

sql_project...a Barns (70)*

```

182      ('2026-01-09', 'Cash', 10),
183      ('2026-01-10', 'Online', 25);
184
185  --section D
186  --Display all products
187  select product_id ,product_name from product_table
188
189  --Display products whose price is greater than 1000.
190  select product_id, product_name, price from product_table where price>1000
191
192  --Find customers from a particular city(eg.LA)
193  select customerid, customername ,adress from customer where adress='LA'
194
195  --Update the price of a product.id=309
196  select * from product_table
197  update product_table set price=1010 where product_id=309
198
199  --Delete customers who have not placed any orders
200  select customer.address, customer.email_id, customer.phoneno, customer.customername, customer.customerid
201  from customer left join order_table on order_table.cid = customer.customerid where order_table.cid is null
202

```

109 % 19 0

Ln: 196, Ch: 1 (85 chars, 2 lines) SPC CRLF Windows 1252

product_id	product_name	price	quantity_in_stock	catid	suplid	wareld
1	301	Laptop	800.00	50	1	201 501
2	302	Office Chair	120.00	100	2	202 502
3	303	T-Shirt	25.00	200	3	203 503
4	304	Rice Bag	40.00	150	4	204 504
5	305	Novel Book	15.00	120	5	205 505
6	306	Football	30.00	80	6	206 506
7	307	Face Cream	18.00	90	7	207 507
8	308	Toy Car	22.00	110	8	208 508
9	309	Car Battery	1010.00	60	9	209 509
10	310	Notebook Pack	10.00	300	10	210 510

Query executed successfully.

MB\SQLEXPRESS01 (17.0 RTM) MB\Monisha Barns (70) smart_retail_project 00:00:00 Row: 1, Col: 1 10 rows

Ready

Object Explorer

Connect ▼ MB\SQLEXPRESS01 (SQL Serv

Databases

Security

Server Objects

Replication

Management

XEvent Profiler

sql_project...a Barns (70)*

```

183      ('2026-01-10', 'Online', 25);
184
185  --section D
186  --Display all products
187  select product_id ,product_name from product_table
188
189  --Display products whose price is greater than 1000.
190  select product_id, product_name, price from product_table where price>1000
191
192  --Find customers from a particular city(eg.LA)
193  select customerid, customername ,adress from customer where adress='LA'
194
195  --Update the price of a product.id=309
196  select * from product_table
197  update product_table set price=1010 where product_id=309
198
199  --Delete customers who have not placed any orders
200  select customer.address, customer.email_id, customer.phoneno, customer.customername, customer.customerid
201  from customer left join order_table on order_table.cid = customer.customerid where order_table.cid is null
202
203  delete customer from customer left join order_table on order_table.cid = customer.customerid where order_table.cid is null

```

109 % 19 0

Ln: 200, Ch: 1 (211 chars, 2 lines) SPC CRLF Windows 1252

address	email_id	phoneno	customername	customerid
---------	----------	---------	--------------	------------

Query executed successfully.

MB\SQLEXPRESS01 (17.0 RTM) MB\Monisha Barns (70) smart_retail_project 00:00:00 Row: 1, Col: 1 0 rows

Item(s) Saved

Section E – Filtering, Sorting & Grouping

The screenshot shows the SQL Server Enterprise Manager interface. The query editor contains the following script:

```
200 select customer.address, customer.email_id, customer.phoneno, customer.customername, customer.customerid
201 from customer left join order_table on order_table.cid = customer.customerid where order_table.cid is null
202
203 delete customer from customer left join order_table on order_table.cid = customer.customerid where order_table.cid is null
204
205 --section E
206
207 --List orders sorted by order date.
208 select * from order_table order by orderdate asc
209
210 --Show total sales for each day.
211 select orderdate, sum(total_amount) as 'Total Amount' from order_table group by orderdate
212
213 --Find total quantity sold for each product.
214 select productid, sum(quantity) as 'Total Quantity' from order_detail group by productid
215
216 --Display customers who placed more than three orders
217 select cid as 'Customer ID', count(orderid) as 'Count' from order_table group by cid having count(cid)>3
218
219 --Section F
220 --Display order details with customer name and product name
```

The Results pane shows the output of the first query:

orderid	orderdate	total_amount	cid
1	2026-01-01	800	101
2	2026-01-02	120	102
3	2026-01-03	50	103
4	2026-01-04	40	104
5	2026-01-05	30	105
6	2026-01-06	18	106
7	2026-01-07	22	107
8	2026-01-08	150	108
9	2026-01-09	10	109
10	2026-01-10	25	110
11	2026-02-13	250	105

Query executed successfully. MB/SQLXPRESS01 (17.0 RTM) | MB/Monisha Barns (70) | smart_retail_project | 00:00:00 | Row: 1, Col: 1 | 11 rows

The screenshot shows the SQL Server Enterprise Manager interface. The query editor contains the following script:

```
200 select customer.address, customer.email_id, customer.phoneno, customer.customername, customer.customerid
201 from customer left join order_table on order_table.cid = customer.customerid where order_table.cid is null
202
203 delete customer from customer left join order_table on order_table.cid = customer.customerid where order_table.cid is null
204
205 --section E
206
207 --List orders sorted by order date.
208 select * from order_table order by orderdate asc
209
210 --Show total sales for each day.
211 select orderdate, sum(total_amount) as 'Total Amount' from order_table group by orderdate
212
213 --Find total quantity sold for each product.
214 select productid, sum(quantity) as 'Total Quantity' from order_detail group by productid
215
216 --Display customers who placed more than three orders
217 select cid as 'Customer ID', count(orderid) as 'Count' from order_table group by cid having count(cid)>3
218
219 --Section F
220 --Display order details with customer name and product name
```

The Results pane shows the output of the second query:

orderdate	Total Amount
2026-01-01	800
2026-01-02	120
2026-01-03	50
2026-01-04	40
2026-01-05	30
2026-01-06	18
2026-01-07	22
2026-01-08	150
2026-01-09	10
2026-01-10	25
2026-02-13	250

Query executed successfully. MB/SQLXPRESS01 (17.0 RTM) | MB/Monisha Barns (70) | smart_retail_project | 00:00:00 | Row: 1, Col: 1 | 11 rows

smart_retail_project

Object Explorer

- Connect
- MB\SQLEXPRESS01 (SQL Serv
 - Databases
 - Security
 - Server Objects
 - Replication
 - Management
 - XEvent Profiler

sql_project...a Barns (70)

```

200 select customer.address, customer.email_id, customer.phoneno, customer.customername, customer.customerid
201 from customer left join order_table on order_table.cid = customer.customerid where order_table.cid is null
202
203 delete customer from customer left join order_table on order_table.cid = customer.customerid where order_table.cid is null
204
205 --section E
206
207 --List orders sorted by order date.
208 select * from order_table order by orderdate asc
209
210 --Show total sales for each day.
211 select orderdate, sum(total_amount) as 'Total Amount' from order_table group by orderdate
212
213 --Find total quantity sold for each product.
214 select productid, sum(quantity) as 'Total Quantity' from order_detail group by productid
215
216 --Display customers who placed more than three orders
217 select cid as 'Customer ID', count(orderid) as 'Count' from order_table group by cid having count(cid)>3
218
219 --Section F
220 --Display order details with customer name and product name

```

109 %

Results Messages

productid	Total Quantity
1	301
2	302
3	303
4	304
5	305
6	306
7	307
8	308
9	309
10	310

Query executed successfully.

MB\SQLEXPRESS01 (17.0 RTM) MB\Monisha Barns (70) smart_retail_project 00:00:00 Row: 1, Col: 1 10 rows

smart_retail_project

Object Explorer

- Connect
- MB\SQLEXPRESS01 (SQL Serv
 - Databases
 - Security
 - Server Objects
 - Replication
 - Management
 - XEvent Profiler

sql_project...a Barns (70)

```

200 select customer.address, customer.email_id, customer.phoneno, customer.customername, customer.customerid
201 from customer left join order_table on order_table.cid = customer.customerid where order_table.cid is null
202
203 delete customer from customer left join order_table on order_table.cid = customer.customerid where order_table.cid is null
204
205 --section E
206
207 --List orders sorted by order date.
208 select * from order_table order by orderdate asc
209
210 --Show total sales for each day.
211 select orderdate, sum(total_amount) as 'Total Amount' from order_table group by orderdate
212
213 --Find total quantity sold for each product.
214 select productid, sum(quantity) as 'Total Quantity' from order_detail group by productid
215
216 --Display customers who placed more than three orders
217 select cid as 'Customer ID', count(orderid) as 'Count' from order_table group by cid having count(cid)>3
218
219 --Section F
220 --Display order details with customer name and product name

```

109 %

Results Messages

Customer ID	Count
-------------	-------

Query executed successfully.

MB\SQLEXPRESS01 (17.0 RTM) MB\Monisha Barns (70) smart_retail_project 00:00:00 Row: 1, Col: 1 0 rows

Section F – Joins & Multi-table Queries

The screenshot shows a SQL Server Enterprise Manager window with a query window titled 'sql_project...a Barns (70)'. The query contains the following SQL code:

```
--Find total quantity sold for each product.
select productid, sum(quantity) as 'Total Quantity' from order_detail group by productid

--Display customers who placed more than three orders
select cid as 'Customer ID', count(orderid) as 'Count' from order_table group by cid having count(cid)>3

--Section F
--Display order details with customer name and product name
select customer.customerid, customer.customername, product_table.product_name, product_table.product_id, order_detail.order_id
from order_table
join customer on order_table.cid=customer.customerid
join order_detail on order_table.orderid = order_detail.order_id
join product_table on order_detail.productid = product_table.product_id

--Display supplier-wise product list.
select product_table.product_id, product_table.product_name, supplier.suppliername
from order_detail
join product_table on order_detail.productid=product_table.product_id
join supplier on product_table.supid = supplier.supplierid
```

The results of the query are displayed in a table with 10 rows and 5 columns:

	customerid	customername	product_name	product_id	order_id
1	101	John Smith	Laptop	301	10
2	102	Sarah Lee	Office Chair	302	11
3	103	David Kim	T-Shirt	303	12
4	104	Emma Brown	Rice Bag	304	13
5	105	Michael Chen	Novel Book	305	14
6	106	Olivia Davis	Football	306	15
7	107	Daniel Wilson	Face Cream	307	16
8	108	Sophia Moore	Toy Car	308	17
9	109	James Taylor	Car Battery	309	18
10	110	Ava Martinez	Notebook Pack	310	19

The status bar at the bottom indicates 'Query executed successfully.' and 'MB\SQLEXPRESS01 (17.0 RTM) MB\Monisha Barns (70) smart_retail_project 00:00:00 Row: 1, Col: 1 10 rows'.

The screenshot shows a SQL Server Enterprise Manager window with a query window titled 'sql_project...a Barns (70)'. The query contains the following SQL code:

```
--Display customers who placed more than three orders
select cid as 'Customer ID', count(orderid) as 'Count' from order_table group by cid having count(cid)>3

--Section F
--Display order details with customer name and product name
select customer.customerid, customer.customername, product_table.product_name, product_table.product_id, order_detail.order_id
from order_table
join customer on order_table.cid=customer.customerid
join order_detail on order_table.orderid = order_detail.order_id
join product_table on order_detail.productid = product_table.product_id

--Display supplier-wise product list.
select product_table.product_id, product_table.product_name, supplier.suppliername
from order_detail
join product_table on order_detail.productid=product_table.product_id
join supplier on product_table.supid = supplier.supplierid

--example of JOINS:
--inner join
```

The results of the query are displayed in a table with 10 rows and 3 columns:

	product_id	product_name	suppliername
1	301	Laptop	TechSource Ltd
2	302	Office Chair	FurniWorld
3	303	T-Shirt	StyleHub
4	304	Rice Bag	FreshMart
5	305	Novel Book	BookPlanet
6	306	Football	SportyPro
7	307	Face Cream	BeautyGlow
8	308	Toy Car	ToyLand
9	309	Car Battery	AutoGear
10	310	Notebook Pack	OfficePlus

The status bar at the bottom indicates 'Query executed successfully.' and 'MB\SQLEXPRESS01 (17.0 RTM) MB\Monisha Barns (70) smart_retail_project 00:00:00 Row: 1, Col: 1 10 rows'.

sql_project...a Barns (70)*

```
232
233
234 --example of JOINS:
235 --inner join: show customers who have placed orders.
236 select customer.customerid, customer.customername, order_table.orderid from customer inner join order_table on customer.customerid = order_table.cid
237
238
239 --left join
240
241
242 --right join
243
244
245 --full join
246
247
248
249 --section G
250 --Find products priced higher than average price
```

109 % 19 0

Ln: 237, Ch: 1 (151 chars, 1 lines) SPC CRLF Windows 1252

	customerid	customername	orderid
1	101	John Smith	10
2	102	Sarah Lee	11
3	103	David Kim	12
4	104	Emma Brown	13
5	105	Michael Chen	14
6	106	Olivia Davis	15
7	107	Daniel Wilson	16
8	108	Sophia Moore	17
9	109	James Taylor	18
10	110	Ava Martinez	19
11	105	Michael Chen	20
12	108	Sophia Moore	21

Query executed successfully. MB\SQLEXPRESS01 (17.0 RTM) MB\Monisha Barns (70) smart_retail_project 00:00:00 Row: 1, Col: 1 12 rows

Ready

sql_project...a Barns (70)*

```
232
233
234 --example of JOINS:
235 --inner join: show customers who have placed orders.
236 select customer.customerid, customer.customername, order_table.orderid from customer inner join order_table on customer.customerid = order_table.cid
237
238
239 --left join: Show all products and their supplier names
240 select product_table.product_id, product_table.product_name, supplier.suppliername from product_table left join supplier on product_table.supid = supplier.supplierid
241
242 --right join:
243
244
245 --full join
246
247
248
249 --section G
250 --Find products priced higher than average price
```

109 % 19 0

Ln: 240, Ch: 1 (168 chars, 1 lines) SPC CRLF Windows 1252

	product_id	product_name	suppliername
1	301	Laptop	TechSource Ltd
2	302	Office Chair	FurniWorld
3	303	T-Shirt	StyleHub
4	304	Rice Bag	FreshMart
5	305	Novel Book	BookPlanet
6	306	Football	SportsPro
7	307	Face Cream	BeautyGlow
8	308	Toy Car	ToyLand
9	309	Car Battery	AutoStar
10	310	Notebook Pack	OfficePlus

Query executed successfully. MB\SQLEXPRESS01 (17.0 RTM) MB\Monisha Barns (70) smart_retail_project 00:00:00 Row: 1, Col: 1 10 rows

Ready

sql_project...a Barns (70)*

```

232
233
234 --example of JOINS:
235 --inner join: show customers who have placed orders.
236 select customer.customerid, customer.customername, order_table.orderid from customer inner join order_table on customer.customerid = order_table.cid
237
238
239 --left join: Show all products and their supplier names
240 select product_table.product_id, product_table.product_name, supplier.suppliername from product_table left join supplier on product_table.supid = supplier.supplierid
241
242 --right join: Show all warehouses and the products stored in them.
243 select warehouse.warehouseid, warehouse.warehouse_name, product_table.product_name from warehouse right join product_table on warehouse.warehouseid = product_table.wa
244
245 --full join
246
247
248
249 --section G
250 --Find products priced higher than average price

```

109 % 19 0

Ln: 243, Ch: 1 (173 chars, 1 lines) SPC CRLF Windows 1252

	warehouseid	warehouse_name	product_name
1	501	Central Warehouse	Laptop
2	502	West Storage	Office Chair
3	503	East Depot	T-Shirt
4	504	South Hub	Rice Bag
5	505	North Storage	Novel Book
6	506	Metro Warehouse	Football
7	507	Prime Depot	Face Cream
8	508	Quick Storage	Toy Car
9	509	City Warehouse	Car Battery
10	510	Global Depot	Notebook Pack

Query executed successfully. MB\SQLEXPRESS01 (17.0 RTM) MB\Monisha Barns (70) smart_retail_project 00:00:00 Row: 1, Col: 1 10 rows

Ready

sql_project...a Barns (70)*

```

235 --inner join: show customers who have placed orders.
236 select customer.customerid, customer.customername, order_table.orderid from customer inner join order_table on customer.customerid = order_table.cid
237
238
239 --left join: Show all products and their supplier names
240 select product_table.product_id, product_table.product_name, supplier.suppliername from product_table left join supplier on product_table.supid = supplier.supplierid
241
242 --right join: Show all warehouses and the products stored in them.
243 select warehouse.warehouseid, warehouse.warehouse_name, product_table.product_name from warehouse right join product_table on warehouse.warehouseid = product_table.wa
244
245 --full join: Show all customers and all payments, whether they match or not.
246 select customer.customerid, customer.customername, payment.paymentid, payment.payment_amount from customer full join payment on customer.customerid = payment_order_id

```

109 % 19 0

Ln: 246, Ch: 1 (169 chars, 1 lines) SPC CRLF Windows 1252

	customerid	customername	paymentid	payment_amount
1	101	John Smith	NULL	NULL
2	102	Sarah Lee	NULL	NULL
3	103	David Kim	NULL	NULL
4	104	Emma Brown	NULL	NULL
5	105	Michael Chen	NULL	NULL
6	106	Olivia Davis	NULL	NULL
7	107	Daniel Wilson	NULL	NULL
8	108	Sophia Moore	NULL	NULL
9	109	James Taylor	NULL	NULL
10	110	Ava Martinez	NULL	NULL
11	NULL	NULL	401	800.00
12	NULL	NULL	402	120.00
13	NULL	NULL	403	50.00
14	NULL	NULL	404	40.00
15	NULL	NULL	405	30.00
16	NULL	NULL	406	18.00
17	NULL	NULL	407	22.00
18	NULL	NULL	408	150.00
19	NULL	NULL	409	10.00
20	NULL	NULL	410	25.00

Query executed successfully. MB\SQLEXPRESS01 (17.0 RTM) MB\Monisha Barns (70) smart_retail_project 00:00:00 Row: 1, Col: 1 20 rows

Ready

Section G – Subqueries & Advanced SQL

SQL Server Enterprise Edition (64-bit) - smart_retail_project

Object Explorer: Connect to MB\SQLEXPRESS01 (SQL Serv) | Databases | Security | Server Objects | Replication | Management | XEvent Profiler

Query Editor: sql_project...a Barns (70) | 109% | 19 | 0 | 0 | 19 | Lnt 251, Cht 1 (111 chars, 1 lines) | SPC | CRLF | Windows 1252

```
--full join
--section G
--Find products priced higher than average price
select price, product_id, product_name from product_table where price>(select avg(price) from product_table)
--Display customers who never placed an order.
select * from customer where customerid NOT IN(select cid from order_table)
--Find top-selling product using subquery.
select productid from order_detail group by productid having sum(quantity) =
(select max(Total_quantity) from
(select productid, sum(quantity) as Total_quantity from order_detail group by productid) as sales)
--Section H
--Create a scalar function to calculate discount.
create function calculate_discount(@num money)
returns money
```

Results: Messages | 1 | price | product_id | product_name | 2 | 1010.00 | 309 | Car Battery

Query executed successfully. | MB\SQLEXPRESS01 (17.0 RTM) | MB\Monisha Barns (70) | smart_retail_project | 00:00:00 | Row: 1, Col: 1 | 2 rows

SQL Server Enterprise Edition (64-bit) - smart_retail_project

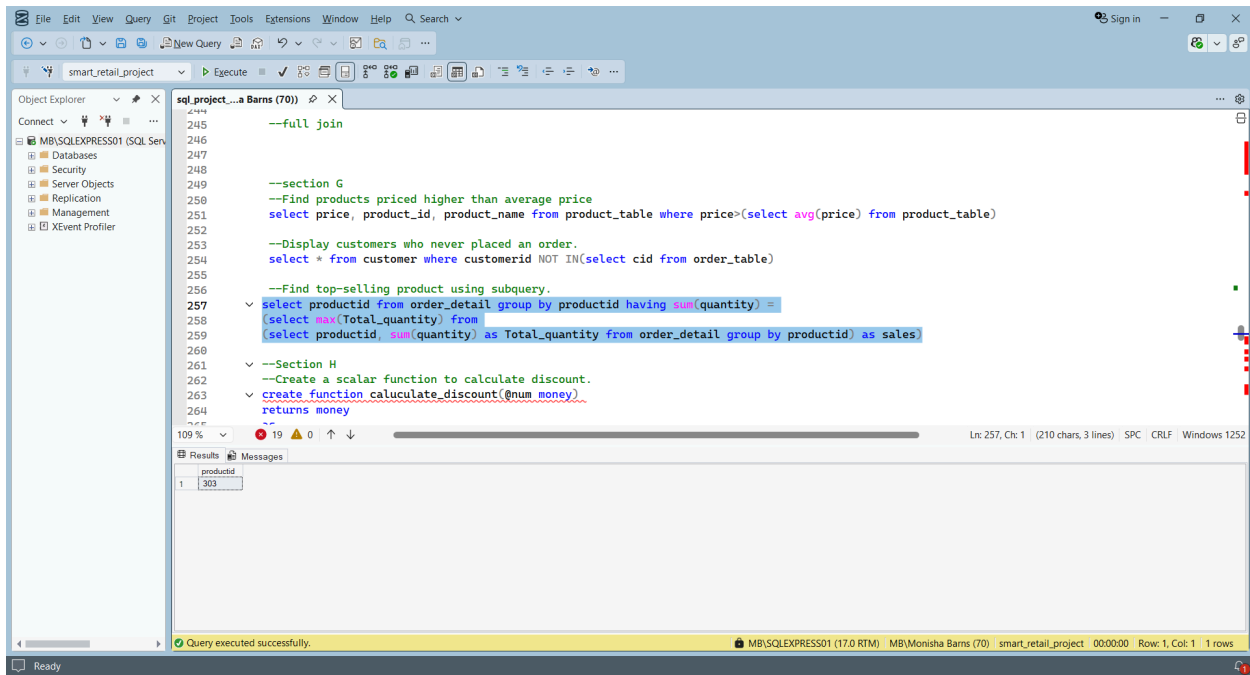
Object Explorer: Connect to MB\SQLEXPRESS01 (SQL Serv) | Databases | Security | Server Objects | Replication | Management | XEvent Profiler

Query Editor: sql_project...a Barns (70) | 109% | 19 | 0 | 0 | 19 | Lnt 255, Cht 1 (78 chars, 1 lines) | SPC | CRLF | Windows 1252

```
--full join
--section G
--Find products priced higher than average price
select price, product_id, product_name from product_table where price>(select avg(price) from product_table)
--Display customers who never placed an order.
select * from customer where customerid NOT IN(select cid from order_table)
--Find top-selling product using subquery.
select productid from order_detail group by productid having sum(quantity) =
(select max(Total_quantity) from
(select productid, sum(quantity) as Total_quantity from order_detail group by productid) as sales)
--Section H
--Create a scalar function to calculate discount.
create function calculate_discount(@num money)
returns money
```

Results: Messages | customerid | customername | phoneno | email_id | address

Query executed successfully. | MB\SQLEXPRESS01 (17.0 RTM) | MB\Monisha Barns (70) | smart_retail_project | 00:00:00 | Row: 1, Col: 1 | 0 rows



Section H – Functions & Stored Procedures

SQL Server Enterprise Edition (64-bit) - smart_retail_project

Object Explorer

- System Tables
- FileTables
- External Tables
- Graph Tables
- dbo.category
- dbo.customer
- dbo.order_detail
- dbo.order_table
- dbo.payment
- dbo.product_table
- dbo.supplier
- dbo.warehouse
- Views
- External Resources
- Synonyms
- Programmability
 - Stored Procedures
 - Functions
 - Table-valued Functions
 - Scalar-valued Functions
 - dbo.calculate_discount
 - Aggregate Functions
 - System Functions
 - Database Triggers
 - Assemblies
 - Types
 - Rules
 - Defaults
 - Sequences
- Query Store
- Service Broker
- Storage
- Security
 - studentdata
- Server Objects

sql_project...a Barns (70)

```
--Section H
--Create a scalar function to calculate discount.
create function caluculate_discount(@num money)
returns money
as
begin
    return
    if @num > 1000
        set @num = @num * 0.10;
    else if @num > 500
        set @num = @num * 0.05;
    else
        set @num = @num * 0.02;
    return @num;
end

select dbo.caluculate_discount(500) as 'To Calculate discount'
```

Results

To Calculate discount
10.00

Messages

Query executed successfully.

MB/SQLEXPRESS01 (17.0 RTM) MB/Monisha Barns (70) smart_retail_project 00:00:00 Row: 1, Col: 1 1 rows

SQL Server Enterprise Edition (64-bit) - smart_retail_project

Object Explorer

- System Tables
- FileTables
- External Tables
- Graph Tables
- dbo.category
- dbo.customer
- dbo.order_detail
- dbo.order_table
- dbo.payment
- dbo.product_table
- dbo.supplier
- dbo.warehouse
- Views
- External Resources
- Synonyms
- Programmability
 - System Stored Procedures
 - Functions
 - Table-valued Functions
 - Scalar-valued Functions
 - dbo.calculate_discount
 - Aggregate Functions
 - System Functions
 - Database Triggers
 - Assemblies
 - Types
 - Rules
 - Defaults
 - Sequences
- Query Store
- Service Broker
- Storage
- Security
 - studentdata
- Server Objects

sql_project...a Barns (70)*

```
else
    set @num = @num * 0.02;
return @num;
end

select dbo.caluculate_discount(500) as 'To Calculate discount'
```

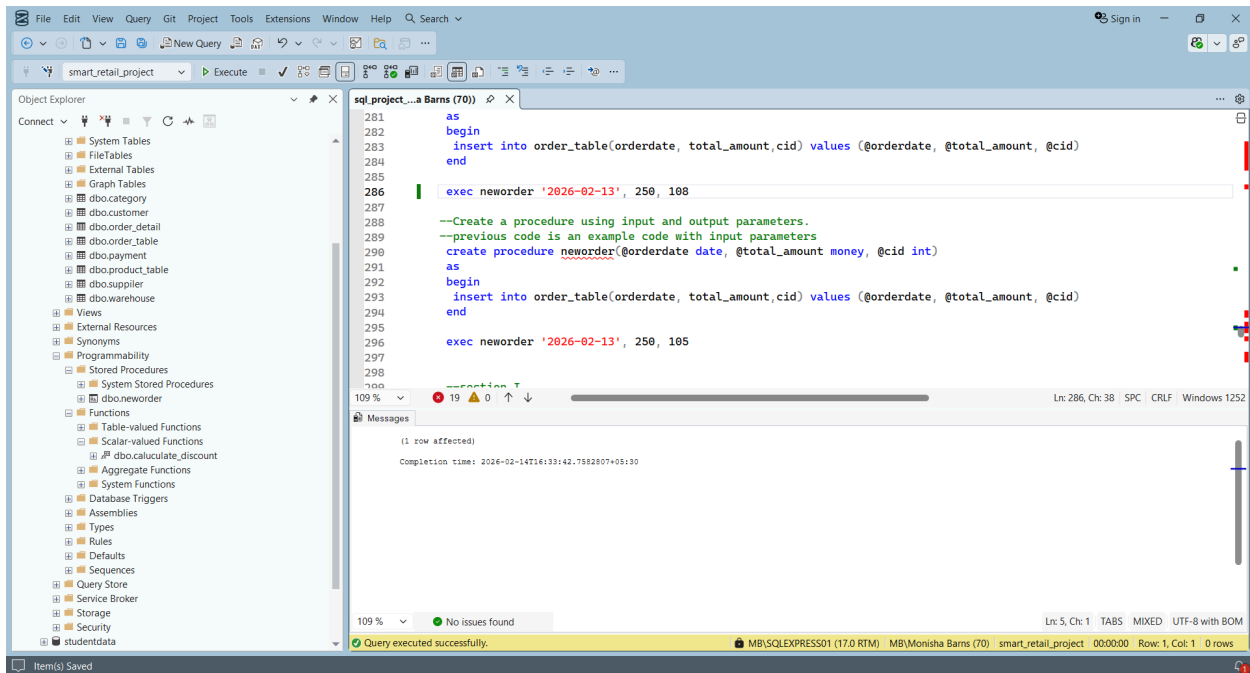
Messages

(1 row affected)

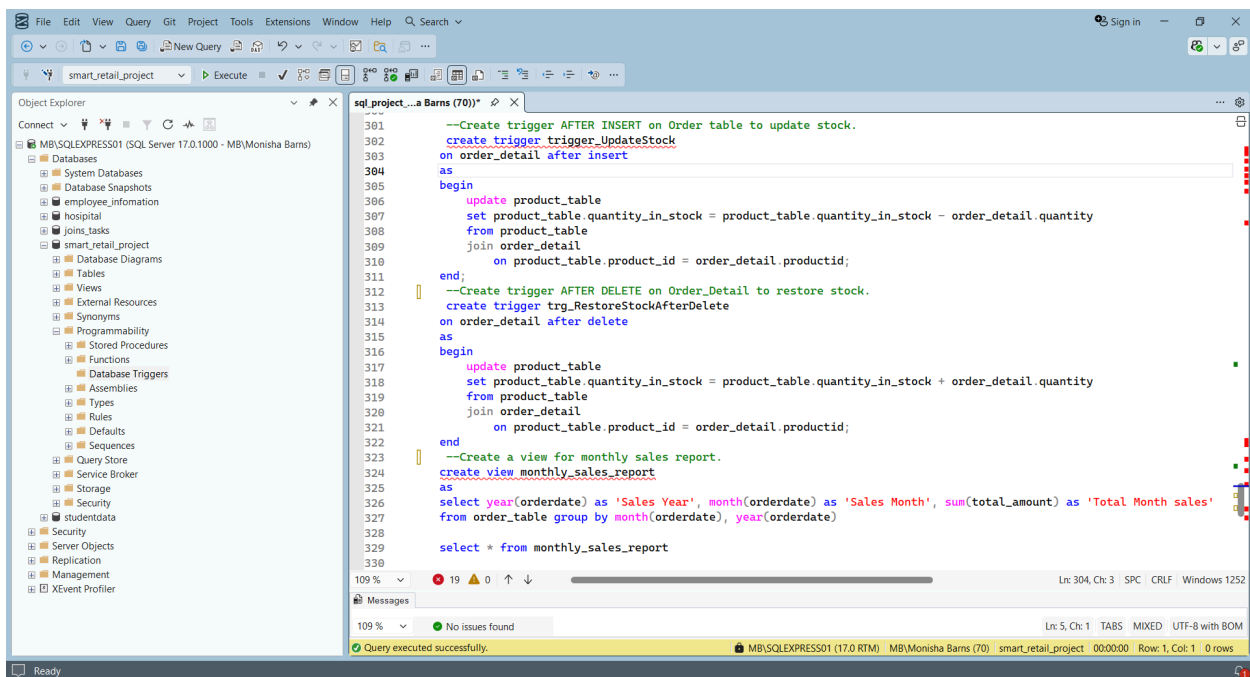
Completion time: 2026-02-14T16:33:42.7582807+05:30

Query executed successfully.

MB/SQLEXPRESS01 (17.0 RTM) MB/Monisha Barns (70) smart_retail_project 00:00:00 Row: 1, Col: 1 0 rows



Section I – Triggers, Views & Index



Section J – AI Integration

The screenshot shows the SQL Server Enterprise Manager interface with a T-SQL script in the main editor. The script includes comments and SQL code for creating a trigger, a view, and an index. The GitHub Copilot Chat interface is open on the right, displaying a validated T-SQL query for showing total sales per product. The chat interface also includes a section for how to modify the query to include the category of each product in the total sales.

```
--Create trigger AFTER DELETE on Order_Detail to restore stock.
create trigger trg_RestoreStockAfterDelete
on Order_Detail after delete
as
begin
    update product_table
    set product_table.quantity_in_stock = product_table.quantity_in_stock + order_detail.quantity
    from product_table
    join order_detail
    on product_table.product_id = order_detail.productid;
end

--Create a view for monthly sales report.
create view monthly_sales_report
as
select year(orderdate) as 'Sales Year', month(orderdate) as 'Sales Month', sum(total_amount) as 'Total Month'
from order_table group by month(orderdate), year(orderdate)

select * from monthly_sales_report

--Create an index on ProductName.
create clustered index productnameindex on product_table(product_name asc)
```

Below is a validated T-SQL query you can run in the **smart_retail_project** database to show total sales per product (quantity and sales amount). It includes products with zero sales.

```
-- Created by GitHub Copilot in SSMS - review carefully;
SELECT
    p.product_id,
    p.product_name,
    COALESCE(SUM(od.quantity), 0) AS total_quantity_sold,
    COALESCE(SUM(od.quantity * CONVERT(DECIMAL(18,2), p.i
FROM dbo.product_table AS p
LEFT JOIN dbo.order_detail AS od
ON od.productid = p.product_id
GROUP BY p.product_id, p.product_name
ORDER BY total_sales_amount DESC;
```

If you want the results limited to products that have sales only, change **LEFT JOIN** to **INNER JOIN**.

Connection: MB\SQLEXPRESS01.smart_retail_project

How can I modify the query to include the category of each product in the total...

Active Document

GPT-5 mini

The screenshot shows the SQL Server Enterprise Manager interface with a T-SQL script in the main editor. The script includes comments and SQL code for creating a database, tables, and a view. The GitHub Copilot Chat interface is open on the right, displaying a validated T-SQL query for showing total sales per product category. The chat interface also includes a section for how to modify the query to include the category of each product in the total sales.

```
create database smart_retail_project

use smart_retail_project

create table category(
    categoryid int primary key identity(1,1),
    categoryname varchar(20),
    description_of_the_product varchar(50))

create table supplier(
    supplierid int primary key identity(201,1),
    suppliername varchar(20),
    phonenumber bigint,
    email varchar(50))

create table customer(
    customerid int primary key identity(101,1),
    customername varchar(20),
    phoneno bigint,
    email_id varchar(50),
    adress varchar(60))

create table order_table(
    orderid int primary key identity(10,1),
    orderdate date,
    total_amount bigint,
    cid int foreign key references customer(customerid))

create table warehouse(
    warehouseid int primary key identity(501,1),
    warehouse_name varchar(30),
```

Below is a validated T-SQL query to get total sales amount for each product category (includes categories with zero sales). Review before executing.

```
-- Created by GitHub Copilot in SSMS - review carefully before executing
SELECT
    c.categoryid,
    c.categoryname,
    COALESCE(SUM(od.quantity * CONVERT(DECIMAL(18,2), p.price)), 0) AS total_sales_amount
FROM dbo.category AS c
LEFT JOIN dbo.product_table AS p
ON p.catid = c.categoryid
LEFT JOIN dbo.order_detail AS od
ON od.productid = p.product_id
GROUP BY c.categoryid, c.categoryname
ORDER BY total_sales_amount DESC;
```

Notes:

- Uses two-part names and validated T-SQL.
- If you only want categories with sales, change the first **LEFT JOIN** to **INNER JOIN** or add **HAVING SUM(od.quantity) > 0**.

Connection: MB\SQLEXPRESS01.smart_retail_project

What is the SQL query to find the total quantity sold for each product?

Active Document

GPT-5 mini

SQL Server Enterprise Edition (64-bit) - SSMS

smart_retail_project

sql_project_a...a Barns (70)

```
1 create database smart_retail_project
2
3 use smart_retail_project
4
5 create table category(
6     categoryid int primary key identity(1,1),
7     categoryname varchar(20),
8     description_of_the_product varchar(50))
9
10 create table supplier(
11     supplierid int primary key identity(201,1),
12     suppliername varchar(20),
13     phonenumber bigint,
14     email varchar(50))
15
16 create table customer(
17     customerid int primary key identity(101,1),
18     customername varchar(20),
19     phoneno bigint,
20     email_id varchar(50),
21     address varchar(60))
22
23 create table order_table(
24     orderid int primary key identity(10,1),
25     orderdate date,
26     total_amount bigint,
27     cid int foreign key references customer(customerid))
28
29 create table warehouse(
30     warehouseid int primary key identity(501,1),
31     warehouse_name varchar(30),
32     )
```

109 %

Messages

100 %

1 in 5 Ch: 1 TABS MIXED UTF-8 with BOM

MB/SQLEXPRESS01 (17.0 RTM) MB/Monisha Barns (70) smart_retail_project 00:00:00 Row: 1, Col: 1 0 rows

Ready

GitHub Copilot Chat

show total sales for each product

sql query to find Top 3 customers

SSMS

1 reference

2 tasks done

GitHub Copilot

Below is a validated T-SQL query to return the top 3 customers by total spend (returns customer id, name, number of orders and total spend). Run in smart_retail_project.

Active Document

Apply

-- Created by GitHub Copilot in SSMS - review carefully before executing

```
SELECT TOP (3)
    c.customerid,
    c.customername,
    COUNT(o.orderid) AS order_count,
    SUM(o.total_amount) AS total_spent
FROM dbo.customer AS c
JOIN dbo.order_table AS o
    ON o.cid = c.customerid
GROUP BY c.customerid, c.customername
ORDER BY total_spent DESC, order_count DESC;
```

If you want ties handled differently (include ties for 3rd place), use FETCH FIRST 3 WITH TIES or rank functions — I can provide that variant if needed.

Connection: MB/SQLEXPRESS01 smart_retail_project

What SQL query can I use to find the total quantity sold for each product?

Active Document

Apply

GPT-5 mini

SQL Server Enterprise Edition (64-bit) - SSMS

smart_retail_project

sql_project_a...a Barns (70)

```
1 create database smart_retail_project
2
3 use smart_retail_project
4
5 create table category(
6     categoryid int primary key identity(1,1),
7     categoryname varchar(20),
8     description_of_the_product varchar(50))
9
10 create table supplier(
11     supplierid int primary key identity(201,1),
12     suppliername varchar(20),
13     phonenumber bigint,
14     email varchar(50))
15
16 create table customer(
17     customerid int primary key identity(101,1),
18     customername varchar(20),
19     phoneno bigint,
20     email_id varchar(50),
21     address varchar(60))
22
23 create table order_table(
24     orderid int primary key identity(10,1),
25     orderdate date,
26     total_amount bigint,
27     cid int foreign key references customer(customerid))
28
29 create table warehouse(
30     warehouseid int primary key identity(501,1),
31     warehouse_name varchar(30),
32     )
```

109 %

Messages

100 %

1 in 5 Ch: 1 TABS MIXED UTF-8 with BOM

MB/SQLEXPRESS01 (17.0 RTM) MB/Monisha Barns (70) smart_retail_project 00:00:00 Row: 1, Col: 1 0 rows

Ready

GitHub Copilot Chat

show total sales for each product

SQL query to find customers who never placed the order

1 reference

Run ValidateGeneratedTSQL

sql_project_accord.sql

Add to new file

Apply

-- Created by GitHub Copilot in SSMS - review carefully before executing

```
SELECT
    c.customerid,
    c.customername,
    c.address,
    c.email_id,
    c.phoneno
FROM dbo.customer AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM dbo.order_table AS o
    WHERE o.cid = c.customerid
)
ORDER BY c.customername;
```

This uses a correlated NOT EXISTS (handles NULLs correctly) and returns meaningful customer columns. If you prefer a LEFT JOIN variant or want to include customers with zero orders plus their order count (0), say so.

Connection: MB/SQLEXPRESS01 smart_retail_project

What SQL query can I use to find the total number of orders placed by each customer?

Active Document

Apply

GPT-5 mini

SQL script for creating a database and tables:

```
1 create database smart_retail_project
2
3 use smart_retail_project
4
5 create table category(
6 categoryid int primary key identity(1,1),
7 categoryname varchar(20),
8 description_of_the_product varchar(50))
9
10 create table supplier(
11 supplierid int primary key identity(201,1),
12 suppliername varchar(20),
13 phonenumber bigint,
14 email varchar(50))
15
16 create table customer(
17 customerid int primary key identity(101,1),
18 customername varchar(20),
19 phonenumber bigint,
20 email_id varchar(50),
21 address varchar(60))
22
23 create table order_table(
24 orderid int primary key identity(10,1),
25 orderdate date,
26 total_amount bigint,
27 cid int foreign key references customer(customerid))
28
29 create table warehouse(
30 warehouseid int primary key identity(501,1),
31 warehouse_name varchar(30),
```

GitHub Copilot Chat interface showing a SQL query suggestion:

SQL query to Show products that are low in stock (less than 20 items remaining).

SSMS

1 reference

2 tasks done

GitHub Copilot

Below is a validated T-SQL query to show products low in stock (less than 20 items). Run in `smart_retail_project`.

sql_project_accord.sql

Created by GitHub Copilot in SSMS - review carefully before executing

```
SELECT product_id,
       product_name,
       quantity_in_stock
FROM dbo.product_table
WHERE quantity_in_stock < 20
ORDER BY quantity_in_stock ASC;
```

If you want the query to include products with exactly 20 items, change the WHERE clause to `<= 20`.

Connection: MB/SQLSERVER01.smart_retail_project

GPT-5 mini

What SQL query can I use to find the total quantity of each product in stock?

Active Document

GPT-5 mini